Brain Dump for Android: System Design

A standalone, phone-only native Android app, with home-screen widgets

Status: **Approved** · Date: 2026-10-10 · Revision 2: **replaces the "website in a frame" design (revision 1). No server, no Vercel, no account. Everything lives on the phone.**

Related: `docs/brain-dump/system-design.md` (the web app, which carries on separately)

> **How to read this doc.** Each section has the plain-English version first. Boxes marked 🎓 **SRE / design lesson** explain *why* a choice was made, using the general idea a site-reliability engineer or system designer would name. They're the "learn while vibing" part.

---

1. The plain-English version

What it is

A real Android app called **Brain Dump** that does what the website does (catch thoughts, sort them into piles, pause and resume tasks), but **entirely on your phone**:

- No sign-in. Open it and dump.
- No internet needed, ever. It works the same on a plane as at home.
- No server, so nothing can go down, and nothing costs money.
- Widgets on your home screen that update **instantly**.

It's a **separate product** from the website. Dumps you make in the app stay in the app; the website keeps its own.

What it can do (version 1)

1. **Dump a thought**: type it, **say it** (recorded, and turned into text on the phone), or **snap a photo** (camera or gallery).
2. **Sort later**: an Inbox with four chips: **Task · Idea · Reminder · Worry**. The app pre-selects its best guess. Dates like "Friday 3pm" are spotted automatically.
3. **Piles**: Tasks, Ideas, Reminders, Worries. Tick off, move, set a date, archive.
4. **Pause / Resume**: a 10-second "here's where I was" voice note before switching tasks, played back when you return.
5. **Widgets**:
   - **Quick Dump (1×1)**: one tap to type.
   - **Dump bar (4×1)**: Type · Voice · Photo · Pause.
   - **Status card (2×2)**: "7 to sort" and "Where you left off: budget sheet".
6. **Shortcuts and sharing**: long-press the app icon for Type/Voice/Photo/Pause; **Share → Brain Dump** from Gallery, Chrome, WhatsApp, etc.
7. **Backup**: automatic backup of your text through your Google account (free), plus **Export everything / Import** to a single file you can keep anywhere.

How it works, as an analogy

The website is like **renting a storage unit across town**: your notes are kept safely elsewhere, you can reach them from any device, but you depend on the storage company being open.

The app is like **a notebook in your pocket**: always there, instant, private, costs nothing, but if you lose the notebook, you lose the notes, unless you photocopied them. That's why backup and export are part of version 1, not an afterthought.

Real-world piece	In the app
The **notebook pages**	An on-phone database (SQLite, via Room)
A **shoebox of photos and tapes**	Photo and voice files in the app's private folder
Sticky tabs on the fridge	Home-screen widgets
Photocopying the notebook	Auto-backup + Export file
A **locked drawer**	Android's app sandbox: other apps can't read Brain Dump's files

What does it cost?

$0. All the tools are free, and there are no running costs because there's no server. You install the app from a file (APK). The Google Play Store (one-time $25) is optional and not planned. iPhone is out of scope ($99/year).

---

2. The big decision: local-first, no server

> 🎓 **Design lesson: every architecture is a set of trade-offs.** A good design doc names what you're giving up, not just what you get.

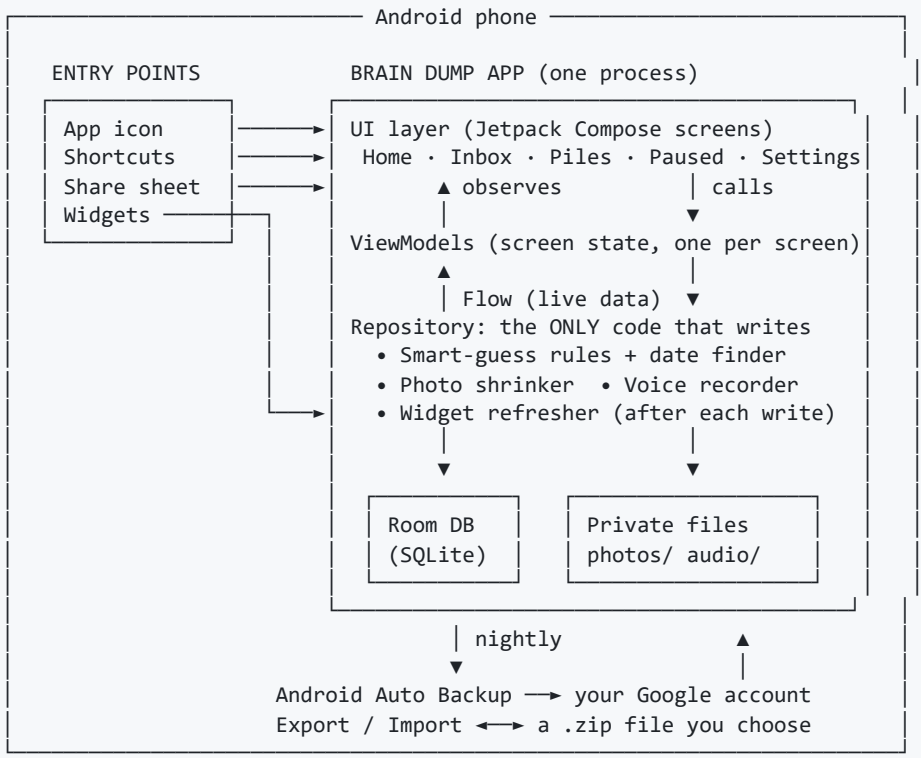
	Website (current)	Phone-only app (this doc)
Works offline	Partly (outbox + cache)	**Always, fully**
Speed	Network round-trips for sorting	**Instant**: everything is local
Running cost	$0 on free tiers, but depends on 3 services	**$0, no services at all**
Things that can go down	Vercel, Neon, Blob, Google sign-in	**Nothing outside the phone**
Privacy	Data on our servers (private, but stored elsewhere)	**Data never leaves the phone** (except your own backups)
Use on laptop + phone	✅	❌ phone only
Lose the phone	Data is safe	**Data lost, unless backed up**
Work to build	Done	Roughly milestones 1–6 again, natively

> 🎓 **SRE lesson: "dependencies are where reliability goes to die."** A system's availability is roughly the *product* of its dependencies' availability. Four services at 99.9% each give at best ~99.6%. The phone-only app has one dependency, the phone, so its reliability is basically the phone's. The cost of removing dependencies is that **durability** (not losing data) moves onto us. That's what §7 is about.

A strong safety property comes free with this design: **the app will not ask for internet permission at all.** Android then blocks it from making any network connection. Your dumps physically can't be sent anywhere by the app. (Backups go through Android itself, under your control.)

---

3. Architecture



🎓 **Design lesson: one writer, many readers.** Only the **Repository** writes data. Screens and widgets only *read* (and ask the repository to change things). So there's exactly one place to get rules right: "save the file before the database row", "refresh widgets after every change", "never lose a dump". This is the same idea as a single service owning its database in a backend system.

🎓 **Design lesson: unidirectional data flow.** Data flows one way: Database → Repository → ViewModel → Screen. User actions flow back the other way as *events*. Screens never hold their own copy of the truth, so the Inbox, the piles and the widgets can't disagree with each other.

Tech choices (all free)

Layer	Choice	Why
Language	**Kotlin**	Google's recommended language for Android
UI	**Jetpack Compose** + Material 3, with Brain Dump's own chunky cream/green theme	Modern, declarative UI (same idea as React): describe the screen for a given state
Database	**Room** (on top of SQLite)	Type-safe queries, live-updating `Flow`s, **tested schema migrations**
Files	App-private storage (`files/photos`, `files/audio`)	Sandboxed by Android; deleted with the app; no storage permission needed
Photos in	System **camera** (`TakePicture`) + system **Photo Picker**	No camera or storage permission needed: Android's own apps handle it
Photo shrink	`ImageDecoder` → resize to 1600 px → **WebP** (quality 80)	Same as the website: ~150–300 KB each; re-encoding drops hidden data incl. **GPS location**
Voice	`MediaRecorder` (AAC/M4A) + Android's **on-device** `SpeechRecognizer`	Free and offline. See the voice note in §8
Smart guess	The website's keyword rules, ported to Kotlin	Same behaviour on both products
Dates	A small Kotlin date finder (today/tomorrow/weekdays/"3pm"/"next week"/"12 Nov")	`chrono-node` is JavaScript-only; we cover the common phrases and test them against the website's examples
Widgets	**Jetpack Glance**	Compose-style widgets; refreshed directly by the repository after each change
Background jobs	**WorkManager**	Nightly clean-up of orphaned files; battery-friendly
Image display	**Coil**	Fast, memory-safe thumbnails from local files
Build	**Gradle** + Android SDK command-line tools + the JDK already on this PC	Free; Android Studio optional (recommended for learning)
CI / releases	**GitHub Actions** (free for public repos)	Every push runs tests; tagging a version builds a signed APK onto a GitHub Release

Layer	Choice	Why
Min / target Android	Android 8.0 (API 26) / Android 15 (API 35)	Covers ~97% of phones

Where the code lives

`apps/brain-dump-android/` in the monorepo. It has no `package.json`, so npm and Turborepo ignore it, and the website's build is unaffected.

---

4. Data model

Same shape as the website, so the two products stay easy to reason about:

```
captures      one per dump
  id (UUID), raw_text, source (text|voice|photo),
  audio_file?, photo_file?, status (inbox|sorted|archived),
  suggested_kind?, created_at, updated_at

items         what a dump was sorted into
  id, capture_id?, kind (task|idea|reminder|worry), title,
  due_at?, done_at?, archived_at?, created_at, updated_at

checkpoints   "Where did I leave off?"
  id, label?, project_tag?, link?, audio_file?, transcript,
  next_step?, created_at, updated_at, resumed_at?, dismissed_at?
```

- No `user_id`: the phone *is* the user.
- Files are stored by **name** (`photos/<capture id>.webp`), not full path, so a restore onto a new phone still works.
- Times are stored as UTC instants; they're shown in the phone's time zone.

> 🎓 **Design lesson: schema migrations are forever.** Once the app is on your phone, its database can't be "reset" without losing data. Every change to the tables ships with a **migration** (old shape → new shape), and Room exports each schema version as a JSON file into the repo so tests can check every upgrade path (v1→v2, v1→v3, …). Migrations are **forward-only**: see "rollback" in §9 for why.
>
> *Built in N5, the first real migration (v1 → v2 adds `checkpoints`):* the tests rebuild a database **exactly as 0.5.0 left it** from the committed `1.json`, fill it with a voice dump, a photo dump and a filed item, upgrade it, and check two things. First, Room agrees the result matches `2.json` column for column. Second, every row is still there. A third test walks **every** old version to the newest, so the list grows as versions are added.
>
> We then broke the migration on purpose (left out one column, `next_step`), and three tests failed with "Migration didn't properly handle: checkpoints". Without them, that bug would have shown up as a crash on your phone the first time you paused something.
>
> Then the real thing: on the emulator, the 0.5.0 app with real data (two photos, a voice note, a filed item) was upgraded to 0.6.0. A full dump of every row **before and after was identical**,

and the new table was there.

Capacity estimate (back-of-the-envelope)

> 🎓 **Design lesson: estimate before you build.** A few lines of arithmetic tell you whether a design will hold.

Heavy use: 20 dumps a day, 5 of them photos, 3 voice notes.

- Photos: 5 × 250 KB = 1.25 MB/day
- Voice: 3 × 10 s × 32 kbit/s ≈ 120 KB/day
- Text: negligible (~10 KB/day)

≈ **1.4 MB/day ≈ 500 MB/year**. Fine on any modern phone. But it's **over** Android Auto Backup's 25 MB limit within weeks, which shapes the backup design in §7. Settings shows "Brain Dump is using 312 MB" so it's never a surprise.

---

5. Key flows

A. Text dump: type → **Dump it** → row saved in the database (a few milliseconds) → Inbox and widgets update instantly.

A2. The half-typed draft (added in N1): the Dump box is saved to disk as you type, and only forgotten once the dump is committed. If saving fails, the text goes back in the box.

> 🎓 **SRE lesson: know which "saved" you mean.** Android offers *saved instance state*, which survives the system killing the app in the background, but is **thrown away** when you swipe the app away or force-stop it. Testing N1 on the emulator showed that gap, so the draft moved to real on-disk storage. Rule of thumb: if losing it would hurt, it goes to disk; screen state is only for things that are cheap to lose (scroll position, which tab is open).

B. Photo dump

1. Tap 📷 → **Take photo** (system camera) or **Choose** (Photo Picker).
2. The app shrinks it to WebP on a background thread, writes it to a temporary file, then **renames** it to `photos/<id>.webp`.
3. Only then is the database row written. The preview plus optional caption shows, and "Saved ✓".

> 🎓 **SRE lesson: write order and atomic renames.** If the phone dies halfway through, what state are we left in?
>
> - File written, row not written → an **orphan file**. Harmless; the nightly clean-up job deletes files no row points to.
> - Row written, file missing → a **broken dump**. Bad. Writing the file *first* makes this impossible.
> - Half-written file → prevented by writing to `*.tmp` and then **renaming**, which the filesystem does in one step (atomically).

> Choosing the order so that every possible crash leaves a *safe* state is a core reliability technique. The clean-up job is a **reconciliation loop**, the same pattern Kubernetes uses: regularly compare "what should exist" with "what does exist" and fix the difference.
>
> *Built in N3:* the clean-up runs in the background every time the app starts. It only touches files **older than an hour**. Without that grace period it could race a save in progress: the file has just been renamed, the row is a millisecond away, and the cleaner sees an "orphan". Running it twice changes nothing (it's *idempotent*). A test breaks the database save *after* the rename and checks that the file goes back to "pending" so you can tap Dump it again.
>
> 🎓 **SRE lesson: the app can die while you're in another app (added in N3).** Tapping **Take photo** hands you to the phone's camera app, and while you're there Android may close Brain Dump to free memory. Cheap phones do this often. When you come back, Android restarts Brain Dump and delivers the photo, but everything that lived only in memory is gone, including *where the camera was told to save the picture*. So that one path is kept in saved state. We tested it by killing the app on the emulator while the camera was open: the photo still arrived on the review screen.

- **No permissions needed.** The camera button uses the phone's own camera app, and **Choose** uses Android's Photo Picker, which hands over only the one photo you pick. The app still has no camera, storage or internet permission.
- **Hidden data is dropped by design.** Shrinking re-draws the picture into a brand-new WebP file, which carries no EXIF metadata, so there is no GPS location to forget to strip. A test plants GPS in a 12-megapixel photo and checks that the result is upright, 1600 px and location-free. On the emulator, a 1.5 MB photo was stored as 236 KB.

C. Voice dump: hold or tap the mic → record (AAC) → stop → saved immediately with "🎙 Voice note" → the phone transcribes in the background (where supported) and fills in the text.

D. Sort: Inbox → tap a chip → in **one database transaction** the item is created and the dump is marked sorted → **Undo** reverses both.

> 🎓 **Design lesson: transactions.** "Create item" and "mark dump sorted" must both happen or neither. Otherwise a crash could leave a dump in the Inbox *and* in a pile. A transaction makes the pair all-or-nothing.
>
> *Built in N2 and proven:* a test makes the second write blow up on purpose and checks that the dump is still safely in the Inbox. We then deleted the transaction for a moment, and that test failed, so we know it really guards the rule. (A test you've never seen fail might not be testing anything.)

D2. Smart guess and the date finder (added in N2): the website reads dates with a JavaScript library (chrono-node), which the phone app can't run. So the phone has a smaller hand-written Kotlin finder for the phrases people actually type ("tomorrow 5pm", "on friday", "in 2 hours", "Oct 12"). A date with no time means 9:00 AM.

> 🎓 **SRE lesson: one contract, two implementations.** When two programs must behave the same, write the expected behaviour down **once** as data, and make both run it. `shared/smart-guess-cases.json` holds about 90 examples (dates, pile guesses, splits).

> The website's test and the phone's test both read that file, and GitHub runs both whenever it changes. If either app drifts, a test fails before you ever see the difference. This is **contract testing**, the same idea teams use between a server and its apps.
>
> Where the phone deliberately differs, it's written down rather than hidden: it ignores "12/10" (12 October in India, 10 December in the US, and a wrong guess is worse than none), it ignores bare "sat"/"sun"/"wed" unless a time or "on" comes with them ("I sat down" isn't a date), and it ignores a bare month name ("march to the shop").

E. Share to Brain Dump: Gallery → Share → Brain Dump → a small "Dumped ✓" sheet → back to where you were. Several photos become one dump each.

F. Widgets: every change in the repository calls "refresh widgets", so the Status card updates **within a second**. No polling and no 30-minute wait.

G. Pause / Resume (built in N5): same as the website. ⏸ (next to 🎙 and 📷, or the **Paused** tab) opens a sheet: an optional voice note, "Where are you up to?", "Next, I need to…", and an optional tag or link. Home shows **Where you left off** with the newest one; **I'm back on it** resumes it (with Undo). After a week, a pause asks "Still relevant?" (**Keep it** / **Let it go** 🌿). If you only spoke, the phone fills in "where you were" from the recording, using the same background queue as voice dumps. Links open in your browser; Brain Dump itself still has no internet permission, and only http/https links are kept.

> 🎓 **SRE lesson: a new owner of files must be taught to the clean-up job (added in N5).** Pause notes keep their recordings in the same `audio/` folder as voice dumps. The clean-up job (§5 B) deletes any file *no database row points at*, and until now only voice dumps pointed at audio files. So without one extra line, **every Pause recording would have been deleted an hour after it was made**, silently, with no crash and no error. A reconciliation loop is only as correct as its idea of "what should exist". There's now a test that runs the clean-up on a Pause recording, and taking that one line out makes it fail.

> 🎓 **Design lesson: a half-written note should survive the phone (added in N5).** You often pause *because* something interrupted you, so you jump to another app, and Android may close Brain Dump while you're there. Everything typed in the sheet, and a finished recording, is kept in **saved state**, so it comes back as you left it. On the emulator: we typed "reply to Sam", pressed Home, killed the app, and reopened it. The sheet was back with the words in it. Swiping the sheet away also keeps your words; only **Cancel** throws them away.

---

6. Reliability goals (SLOs)

> 🎓 **SRE lesson: SLIs, SLOs and error budgets.**
>
> - An **SLI** (indicator) is something you can measure, e.g. "time from tapping Dump to *Saved ✓*".
> - An **SLO** (objective) is the target, e.g. "99% under 300 ms".
> - The **error budget** is the allowed misses (the other 1%). Spend it on shipping features; if you blow it, stop and fix reliability first.
>
> Even a single-user app benefits: SLOs turn "it feels slow" into a number you can test.

What matters	SLI	SLO (target)	How we check
Capture feels instant	Tap Dump → "Saved ✓" (text)	99% < 300 ms	Timed in automated tests + on-device timer
Photo capture	Photo chosen → "Saved ✓"	95% < 2 s	Test with a 12-megapixel photo
Never lose a dump	Dumps confirmed "Saved ✓" that later go missing	**0, ever**	Crash-during-save tests (§9)
App opens fast	Cold start → Home visible	95% < 1 s	Android's startup measurement; **first real reading (v0.1.0, your phone): under 0.5 s** ✅. The emulator (no GPU) shows 1.4–2.5 s, so phone numbers are the ones that count
Widgets are fresh	Data change → widget updated	99% < 2 s	Test: dump, then read widget
Doesn't crash	Crash-free sessions	≥ 99.5%	Local crash log (§8)
Backups exist	Age of newest backup	Auto-backup < 48 h; Settings nags if last export > 30 days	Shown in Settings

---

7. Backup and restore (durability)

> 🎓 **SRE lesson: RPO and RTO.**
>
> - **RPO (Recovery Point Objective)**: how much recent data you can afford to lose. "Backed up nightly" means up to 24 h of dumps could be lost.
> - **RTO (Recovery Time Objective)**: how long getting back to normal takes.
> - And the golden rule: **a backup you've never restored is not a backup.** We test restores.

Three layers, cheapest first:

Layer	Covers	RPO	RTO	Cost
1. Android Auto Backup to your Google account	Database + settings (text, piles, checkpoints), **not** photos/audio (they'd break the 25 MB limit)	~24 h (Android backs up roughly daily on Wi-Fi while charging)	Minutes: reinstall the app on the same Google account and it restores	Free
2. Phone-to-phone transfer (when you get a new phone)	Everything, including photos/audio (no size limit)	0 at transfer time	Part of phone setup	Free
3. Export everything (Settings → Export)	Everything: one `.zip` with the database + photos + audio + a readable `dumps.json`	Whenever you last exported	Minutes: Settings → Import	Free; you choose where to keep it (Drive, PC, …)

- **Import** checks the file (version, manifest, file checksums) **before** touching anything, then replaces data in one go. A broken or half-copied export file can't damage what's on the phone.
- Settings shows **"Last export: 12 days ago"** and gently reminds you after 30 days. That's the ADHD-friendly version of a backup policy.
- **Restore drill** (part of testing): export on the emulator → wipe → import → compare every row and file checksum.

> 🎓 **Design lesson: export formats are a contract.** The export file has a `format_version`. Future versions of the app must keep importing old exports, so you never lose access to an old backup.

---

8. Observability without a server

> 🎓 **SRE lesson: you can't fix what you can't see.** Backend systems send logs and metrics to a monitoring service. We have no server, and deliberately no internet permission, so observability stays **on the phone, under your control**.

- **Event log**: a small rolling log (last ~1,000 events: "photo saved in 640 ms", "migration 1→2 ok", "widget refreshed") stored on the phone. **It never contains what you dumped**, only what happened and how long it took.
- **Crash log**: if the app crashes, the stack trace is saved and shown next launch: "Brain Dump crashed last time. Share the report?" Sharing goes through Android's normal share sheet (e.g. email it to yourself). You decide.
- **Diagnostics screen** (Settings → About → tap version 7×): SLI numbers (e.g. save time p50/p95), storage used, database version, last backup/export, counts.
- **No analytics, no Firebase, no Crashlytics.** They'd need internet and would send data to Google.

> 🎓 **Design lesson: privacy by architecture, not by promise.** "We don't send your data" is a promise. "The app has no internet permission" is a property Android enforces.

Voice transcription (a known unknown)

Android's built-in speech recognizer usually listens to the microphone itself, which clashes with *our* recording (the same problem the website hit on phones). On **Android 13+** it can transcribe from a recorded file, and the phone needs an offline speech pack. **Plan:** a short **spike** (time-boxed experiment) at the start of the voice milestone. If on-device transcription isn't reliable on your phone, voice dumps are still saved and playable; you just type a title.

> 🎓 **Design lesson: spike the riskiest unknown first.** Don't design a whole feature around an API you haven't tried. A one-hour experiment beats a week of rework.

Voice transcription: what the N4 spike found

A time-boxed experiment on the emulator (Android 15), feeding a recorded test sentence to Android's **on-device** recognizer:

1. **It works, fully offline.** "Remember to buy milk tomorrow. Also call the dentist on Friday at 5" came back word for word, from an AAC `.m4a` file like the ones our recorder makes. The recognizer runs in Google's *on-device* speech service. We call `createOnDeviceSpeechRecognizer`, never the default one, which may send audio to a server.
2. **Two recognizer sessions at once break each other** (error 11, "server disconnected"). So the app transcribes **one note at a time**.
3. **Long notes come back in pieces** (roughly one per sentence). We ask for a *segmented session* and stitch the pieces together.
4. **Feeding audio much faster than real time silently drops sentences.** The second sentence vanished at full speed. 4× worked, so we use **2×** for headroom on slower phones. A 10-second note takes about 5 seconds.
5. **It needs Android 13+ and an installed offline language pack.** If your language isn't installed, the app uses another installed variant of the same language (en-IN → en-US). Otherwise the note stays a playable recording titled "🎙 Voice note".

> 🎓 **SRE lesson: spikes pay for themselves.** Three of those five findings (sessions clashing, the pieces, the speed limit) aren't in Android's documentation. Each would have been a confusing "sometimes the words are missing" bug report after release. An hour of experiment turned them into design decisions.

> 🎓 **SRE lesson: a durable work queue (N4).** A voice note is **saved first** (9 ms) and transcribed afterwards. The note's ID is written to disk before transcription starts and removed only when it ends. If the app is closed halfway, the next start finds the ID and finishes the job: *at-least-once* processing, like a server's job queue. Failures are retried, but at most 3 times, so one bad recording can't retry forever. The finished words only replace the "🎙 Voice note" placeholder (*compare-and-set*), so they never overwrite a title you typed yourself. Tested on the emulator by restarting the app mid-queue: the words appeared after the restart.

> 🎓 **Design lesson: graceful degradation.** Each piece can fail on its own without taking the others down. No offline speech pack: you still get a playable recording. Mic permission refused: the sheet explains and offers a shortcut to Settings, with no nagging. App sent to the background mid-note: Android cuts the mic for background apps, so the app saves what it has instead of recording silence.

> 🎓 **Testing lesson: real data finds real bugs.** The first real transcript ("…milk tomorrow … Friday at 5") showed the phone's date finder gluing "tomorrow" to "at 5" across two sentences and getting 5 AM. The website (chrono) doesn't do that. Six new two-date sentences went into the shared test file, both apps now pass all 95 cases, and the drift can't come back unnoticed.

---

9. Testing, releases and rollback

Testing pyramid

Level	What	Examples
Unit (many, fast)	Pure logic	Smart-guess rules, date finder (using the website's examples as shared test cases), export manifest checks
Integration	Real database + files on the emulator	Repository writes, transactions, **every migration path**, export → import round-trip
UI	Compose UI tests on the emulator	Dump → Inbox → sort → pile → undo
Fault injection	Deliberately breaking things	Kill the app mid-photo-save; fill the storage; deny mic permission; corrupt an export file; rotate the screen mid-recording
Manual	Your phone	A short checklist per release (widgets, share, camera, voice on your real device)

> 🎓 **SRE lesson: test the failure paths, not just the happy path.** Most real outages come from situations nobody tested: full disks, half-finished writes, unexpected restarts. Fault injection makes them boring.

Release pipeline

```
push to main ──► GitHub Actions: build + unit tests + lint
tag v1.2.0   ──► GitHub Actions: build + all tests + sign ──► GitHub Release (APK attached)
                                                                      │
                        you, on the phone: open the release page ─────┘──► install update
```

- **Versioning:** `versionName` = `1.2.0` (for humans); `versionCode` = an always-increasing number (for Android).
- **Signing key:** created once, kept in `D:\Metamorphosis\keys\` (never in the repo) **plus a backup copy you keep elsewhere**. GitHub Actions gets it as an encrypted secret. Android only installs an

update signed with the **same key**. Lose it, and updates are impossible without uninstalling, which loses data.
- **Walking skeleton first:** the very first milestone builds an almost-empty app *through the entire pipeline* (code → CI → signed APK → your phone). Every later feature then rides a pipeline that already works.

Rollback (and why we "roll forward")

> 🎓 **SRE lesson: know your rollback story before you need it.** On a server, a bad release is rolled back in seconds. On Android, **installing an older version over a newer one is blocked** (a lower `versionCode`), and even if forced, an older app can't read a database a newer migration already changed. So the plan is **roll forward**: fix the bug and ship v1.2.1 quickly. That's why the release pipeline must be fast and boring, and why migrations get the heaviest testing. Safety net: the app makes an **automatic local copy of the database before running any migration**, kept until the next successful launch.
>
> *Built in N5:* before Room opens the database, the app reads the version stamped in the file. If it's older than the app, it copies the file aside as `brain-dump.db.before-v1`. The copy goes to a temporary name first and is then renamed, the same atomic-rename trick as photos, so a half-finished copy never looks complete. A second attempt at the same upgrade **never overwrites** the first copy, because the first one is the true original. The copy is deleted at the first launch that finds the database already up to date, which proves the upgrade worked.
>
> One thing the emulator taught us: on the real 0.5.0 data, the main database file was only **4 KB**, while the dumps sat in the **write-ahead log** (`-wal`, 120 KB) waiting to be merged. A copy of just that file could have been almost empty. So the copy step also copies the log whenever one still holds changes. On the emulator the log had already been merged by the time of the copy, and the copy was checked by opening it: version 1, all 3 dumps and the filed item.

---

10. Privacy and security

- **No internet permission.** The app cannot send data anywhere.
- All data in **app-private storage**, which other apps can't read. On Android 10+ it's also encrypted at rest by the phone.
- Photos lose hidden data (EXIF, including **GPS**) when shrunk.
- Permissions asked: **microphone** only (when you first record). No camera permission (the system camera app does the capture), no storage permission (the Photo Picker hands over only the photo you chose).
- The **Status card hides "where you left off" text by default** (anyone can glance at a home screen); you can switch it on.
- **Backups:** Auto Backup is end-to-end encrypted when your phone has a screen lock (Android 9+). Exports are plain `.zip` files; Settings warns that they're unencrypted and should be kept somewhere private.
- **Delete everything** in Settings wipes the database and all files.

- The repo is public: no secrets in the Android project; the signing key lives only on your PC and in GitHub's encrypted secrets.

---

11. Risks and open questions

Risk	Likelihood / impact	Plan
Phone lost or broken	Medium / **high**	Auto Backup (text) + Export (everything) + monthly export reminder
Signing key lost	Low / high	Two copies (PC + one you keep elsewhere); documented in the README
Bad migration corrupts data	Low / **high**	Every path tested; pre-migration local copy; forward-only fixes
On-device transcription doesn't work on your phone	Medium / low	Spike first; voice still saved and playable
Date finder misses phrases chrono-node catches	Medium / low	Only a suggestion; you can set the date by hand; grow the test list over time
Storage fills up	Low / medium	Storage use shown in Settings; photos are shrunk; Android's "storage full" errors handled without losing the text part
Building Android takes longer than expected	Medium / low	Ships in small milestones, each one usable on its own
Two separate Brain Dumps (web + app) get confusing	Medium / low	Different icon tint in the app; a sync bridge (e.g. Export → web import) could come later, if wanted
iPhone	n/a	Out of scope

---

12. Build milestones

Each one ends with a **signed APK on your phone** and a short "what we learned" note.

#	Milestone	You can…	SRE / design focus
N0 ✅	**Walking skeleton**: install Android SDK + emulator, empty app with theme and icon, GitHub Actions CI, signing key, first GitHub Release	Install "Brain Dump" and see the Home screen	Release pipeline, signing, versioning
N1 ✅	**Text dumps + Inbox**: Room database v1, repository, Dump box, Inbox, clear/undo	Dump text and see it in the Inbox	Data layer, unidirectional flow, first SLO timer
N2 ✅	**Sorting + piles**: chips, smart guess, date finder, split, piles, done/move/date/archive	Sort into piles	Transactions, shared test cases with the web
N3 ✅	**Photo dumps**: camera, picker, shrink, thumbnails, full-screen view	Dump photos	Write order, atomic rename, reconciliation job
N4 ✅	**Voice dumps**: spike, then recorder + playback + transcription	Dump by voice	Spikes, permissions, graceful degradation
N5 ✅	**Pause / Resume**	Save and resume your place	First real schema migration (v1→v2) + migration tests
N6	**Widgets, shortcuts, Share to Brain Dump**	Use all three widgets and the share sheet	Event-driven updates, multiple entry points
N7	**Backup, export/import, diagnostics**	Export, wipe, import, all back	RPO/RTO, restore drills, on-device observability
N8	**Polish**: search, delete everything, settings, empty states, accessibility	Daily-drive it	Performance budget (cold start), crash-free rate

The **web app's milestone 7** (photo dumps, shortcuts, share) is independent and can run alongside, since the two share no code or data.